计算机学院 操作系统 课程实验报告

实验题目： 实验 1 进程控制		学号：202200130077
日期：2024.3.11	班级：智能班	姓名：薛润泽
Email：		

实验方法介绍：

使用虚拟机环境下的 Ubuntu22.04 环境，编写 c 语言程序，并配合在终端中运行，实现子进程的创建、子进程与父进程的信息交互和进程控制。

实验过程描述：

一、示例实验描述：

1 打开终端命令行窗体，使用 mkdir 新建文件夹后，建立 pctl.c C 语言程序，pctl.h 头文件和 Makefile 项目管理文件。并在命令行中运行 gmake 命令。

Open ~/Documents/OSex1 pctl.c Save

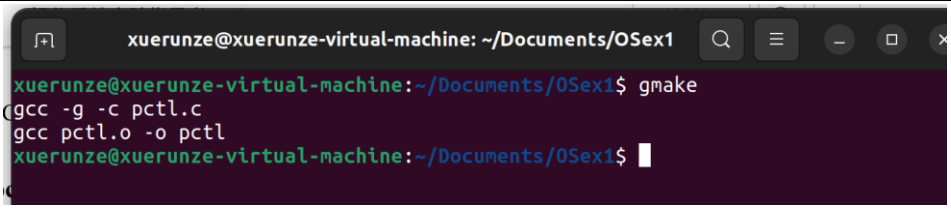
```c
/*
* Filename: pctl.c
* copyright : (C) 2006 by zhanghonglie
* Function: 父子进程的并发执行
*/
#include "pctl.h"
int main(int argc, char *argv[]) {
//如果在命令行上没输入子进程要执行的命令 //则执行缺省的命令
int i;
int pid; //存放子进程号
int status; //存放子进程返回状态
char *args[] = {"/bin/ls","-a",NULL}; //子进程要缺省执行的命令
signal(SIGINT,(__sighandler_t)sigcat); //注册一个本进程处理键盘中断的函数
perror("SIGINT"); //如果系统调用 signal 成功执行，输出"SIGINT"，否则，
//输出"SIGINT"及出错原因
pid=fork() ; //建立子进程
if(pid<0) // 建立子进程失败?
{
printf("Create Process fail!\n");
exit(EXIT_FAILURE);
}
if(pid == 0) // 子进程执行代码段
{
//报告父子进程进程号
printf("I am Child process %d\nMy father is %d\n",getpid(),getppid());
pause(); //暂停，等待键盘中断信号唤醒
//子进程被键盘中断信号唤醒继续执行
printf("%d child will Running: \n",getpid()); //
if(argv[1] != NULL){
//如果运行该程序是用命令行参数指定了子进程要执行的命令
```

Loading file "/home/xuerunze/Documents/OSex1/pctl.c"... C Tab Width: 8 Ln 8, Col 33 INS

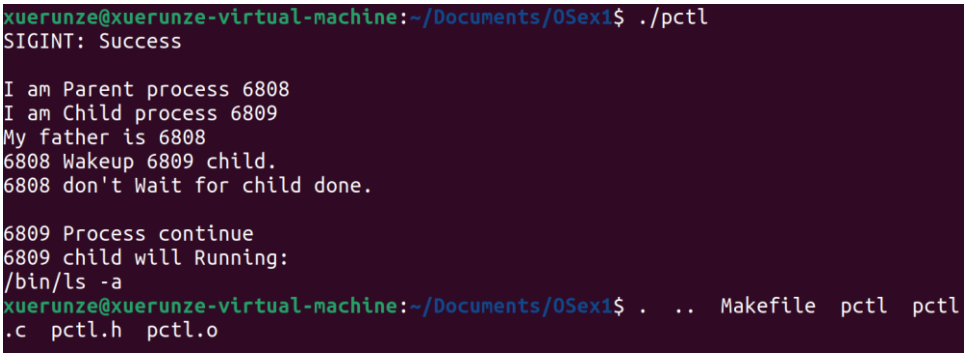
Open ~/Documents/OSex1 pctl.h Save

pctl.c pctl.h

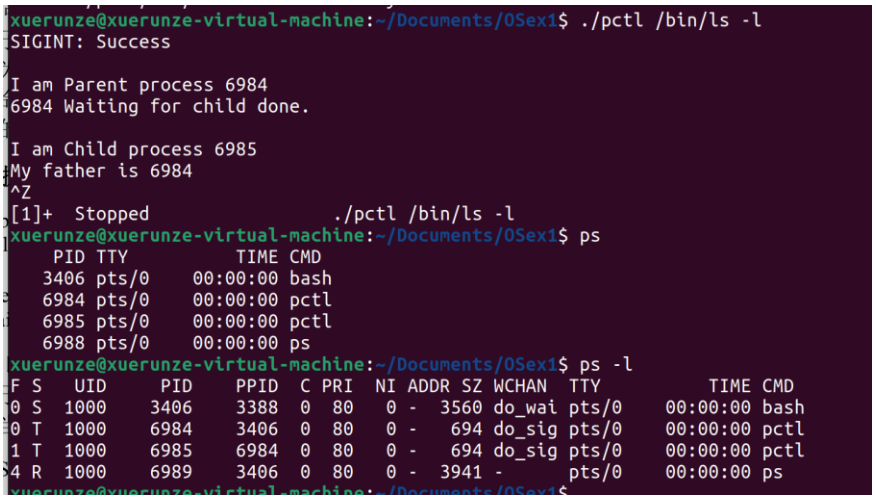
```c
#include <sys/types.h>
#include <wait.h>
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h> //进程自定义的键盘中断信号处理函数 typedef void (*sighandler_t) (int);
void sigcat(){
printf("%d Process continue\n",getpid());
}
```



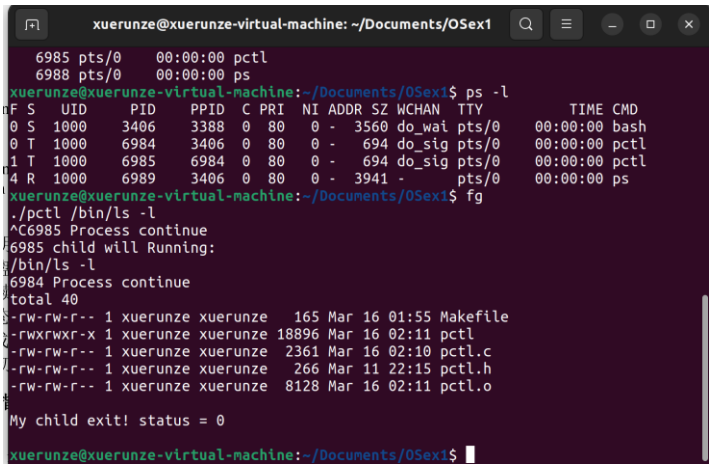
2 执行 pctl 程序，运行结果如下图。



3 再次执行 pctl 程序，并使用 ctrl+z 将当前进程放入后台后，输入 ps 命令查看当前系统进程信息。



再使用 fg 命令，使 pctl 返回前台。



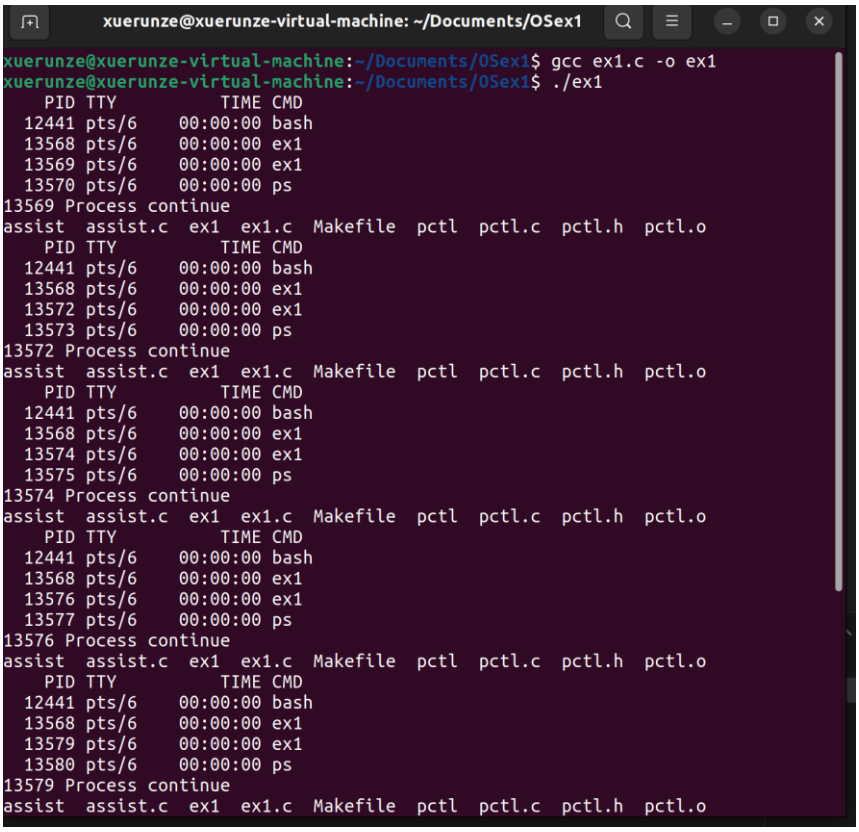
子进程捕捉到键盘中断信号后继续执行制定命令，输出目录中文件名。父进程在接收到子进程执行结束的信号后将清理子进程的退出状态并继续执行，直到

结束。

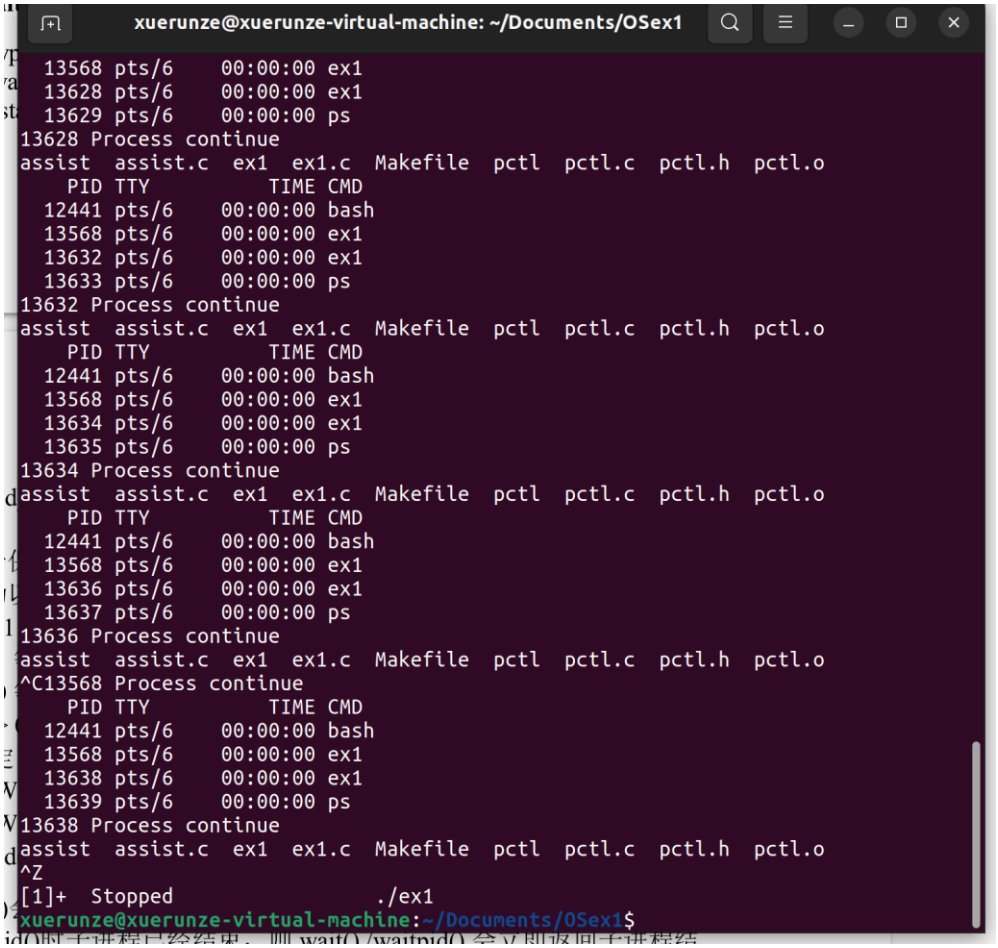
二、独立实验描述

独立实验思路：实验要求我们编写一个多进程并发执行程序。父进程每 3 秒重复建立两个子进程。首先创建的子进程执行 ls 命令，后创建的进程执行 ps 命令，但是首先创建的子进程要晚于后创建的子进程执行相关命令。分析要求可知，要使得父进程每 3 秒钟重复建立两个子进程，可以使用 sleep 函数和 while 循环语句，子进程执行 ls 命令和 ps 命令可以使用 execvp 函数来调用相关命令。其中特别强调了早创建的子进程要晚执行命令，所以可以使用 pause 函数使得早创建的子进程暂停执行，直到接收到晚创建的命令执行完毕的信号再继续自己的命令执行。因为 execvp 函数成功执行时不会有返回值，所以我们把“完毕信号”定为整个晚创建的进程返回时，也就是要在父进程中接受信号。我们可以在父进程中使用 waitpid 函数来等待着晚子进程的执行结束，接收到相关信号后，使用 kill 函数将定义好的键盘中断信号 SIGINT 传送给早子进程，使得早子进程脱离 pause 状态，并进行其程序的正常执行。

下图为程序运行状态：



因为使用了 while(1)循环，所以强制退出时才能结束，如下图：



问题回答：

示例中的 sleep(5)使得唤醒信号出现在 pause 信号之后，而避免了唤醒信号出现在 pause 信号之前造成无法唤醒的情况。

结论分析：

进程的特性：

1、父进程可以和多个子进程一共运行，具有并发性。

2、子进程在执行完毕后会动态消亡，具有动态性

3、子进程在被创建后独立运行，具有独立性，异步性

4、子进程可以和父进程之间进行信息交互，具有交往性

进程的生命期指的是从进程创建到进程结束的整个过程。

进程的实体主要包括程序段，数据段和程序控制段。

程序的状态控制主要涉及对进程的调度，管理和资源分配。

子进程是由父进程通过 fork 系统函数来创建的，其数据与父进程相同，执行与父进程相同的程序。

信号的机理包括产生、发送、接收处理等。

信号实现进程控制主要通过捕捉、忽略、阻塞、同步等完成。

结论：要深入了解学习很重要的一点是搞清楚信号的传递，独立实验要求了先创建的子进程要后执行指令，然而，怎么样向“先子进程”中传递“后子进程”的消息？最开始一直想要从 execvp 函数方向入手，虽然已知其正确执行时不会有返回值，且不会执行往后的程序。但是当调转思考方向，不考虑怎么样从子进程向子进程传递消息，而是考虑怎么从父进程接受子进程消息，再从父进程传递子进程消息时，思路就开朗了。